MACHINE LEARNING · WEEK 3 · LECTURE NOTES FOR STUDENTS

Classification with Probabilities

Logistic regression · maximum likelihood · multiclass classification · Naive Bayes — full explanations, worked examples and practice exercises

Item	Details
Programme	Fall Semester 2026–2027
Syllabus	Week 3: logistic regression, maximum likelihood estimation, multiclass classification, Naive Bayes
Lecturer	Behnam Kiani
Prerequisites	Lecture 2: empirical risk minimisation, linear regression, gradient descent, ridge and lasso. Derivatives of e^x and $\log x$; the chain rule; basic probability.
Companion materials	Lecture 3 slides and videos · Lab 3 handout and notebook
Data used	Palmer penguins · Wisconsin breast cancer (569 biopsies) · SMS Spam Collection (5,572 messages)

How to read these notes

These notes explain everything from the lecture in full sentences, at your own pace. They follow the order of the slides. Every new idea is introduced in four steps: the **intuition** in words, the **mathematics**, a **worked example with real numbers**, and what it means **in practice**. Coloured boxes help you find things quickly:

Box	What it contains
Definition	A precise statement you should be able to reproduce in the exam.
Worked example	A calculation with real numbers, step by step. Redo it with a pen.
Key idea	The one thing to remember from a section.
Common mistake	Errors students make every year — avoid them.
Try it yourself	A short question to check your understanding. Answers are in Section 9.

All numbers in these notes were computed from the real data sets with the code of Lab 3 (fixed random seeds), so you can reproduce every one of them.

Learning outcomes

After studying these notes you should be able to:

1. explain why linear regression is a poor tool for classification and what the sigmoid function fixes;
2. write the logistic regression model, interpret its coefficients as odds ratios, and describe its decision boundary;

3. state the maximum likelihood principle, derive the log loss from a Bernoulli model, and derive its gradient;
1. extend classification to K classes with one-vs-rest and with softmax regression and the cross-entropy loss;
2. apply Bayes' theorem, build Gaussian and multinomial Naive Bayes classifiers, and explain Laplace smoothing;
3. choose a decision threshold from the costs of errors, read a confusion matrix, and compare generative and discriminative models.

Contents

1. From regression to classification
2. Logistic regression
3. Maximum likelihood estimation
4. Multiclass classification
5. Naive Bayes
6. Classification in practice
7. Summary and formula sheet
8. Glossary
9. Practice exercises with solutions
10. Further reading

1. From regression to classification

In Lecture 2 we predicted **numbers**: the body mass of a penguin, the progression of a disease. Many of the most important problems in machine learning are different: we want to predict a **category**. Is this e-mail spam or not? Is this tumour benign or malignant? Which species is this penguin? Problems like these are called **classification** problems, and the models that solve them are called **classifiers**.

1.1 What is a classification problem?

DEFINITION — CLASSIFICATION

We observe examples (x, y), where $x \in \mathbb{R}^d$ is a vector of features and y is a **class label** from a finite set. In **binary** classification $y \in \{0, 1\}$; in **multiclass** classification $y \in \{1, \ldots, K\}$. A **classifier** is a function h that maps x to a class.

By convention, in binary problems we call class 1 the **positive class** — the thing we are looking for (spam, malignant, fraud) — and class 0 the **negative class**. "Positive" does not mean "good": a positive cancer test is bad news.

Classification is everywhere. Some examples, with the kind of model from this week that is often used, at least as a first baseline:

Table 1. Classification in the real world.

Application	Features x	Label y	Typical model
Spam filtering	word counts of the message	spam / not spam	Naive Bayes
Medical diagnosis	measurements from a biopsy image	malignant / benign	logistic regression
Credit scoring	income, debts, repayment history	defaults / repays	logistic-regression scorecard
Fraud detection	amount, place, time of a payment	fraud / genuine	logistic regression (very imbalanced)
Ad-click prediction	user, page, ad (millions of sparse features)	click / no click	large-scale logistic regression
Image recognition	pixels (or learned features)	one of K objects	softmax (last layer of a neural network)

1.2 Predict probabilities first, decide second

The natural loss for classification is the **0–1 loss**: 1 if the prediction is wrong, 0 if it is right. Its average over a data set is the **error rate**, which is 1 − accuracy. Unfortunately, as we saw in Lecture 2, the 0–1 loss is flat almost everywhere: a small change in the parameters usually changes no prediction at all, so its gradient is zero and gradient descent cannot use it.

The central idea of this week is therefore: **do not predict the class directly; predict the probability of each class**, $P(y = 1 \mid x)$, and only then turn the probability into a decision with a **threshold** t:

$$\mathbf{x \rightarrow \text{model} \rightarrow P(y = 1 \mid x) \rightarrow \text{compare with } t \rightarrow \hat{y} \in \{0, 1\}}$$

Why is a probability more useful than a label? Three reasons:

- **Confidence.** P(spam) = 0.51 and P(spam) = 0.99 are both classified as spam with t = 0.5, but they are very different situations. The label throws this information away.
- **Costs.** Different mistakes have different costs. Missing a cancer is far worse than a false alarm. With a probability we can choose the threshold to match the costs (Section 2.7).
- **Ranking.** We can sort cases by probability: review the 100 most suspicious payments first, or call the 50 customers most likely to leave.

KEY IDEA

A good classifier is a good **probability model**. The decision (the threshold) is a separate step that depends on what each mistake costs.

1.3 Why not just use linear regression?

The first idea that comes to mind is to code the classes as 0 and 1 and fit last week's least-squares line, predicting class 1 when the line is above 0.5. Figure 1 tries this on penguins: Adelie (y = 0) against Chinstrap (y = 1), using bill length as the only feature.

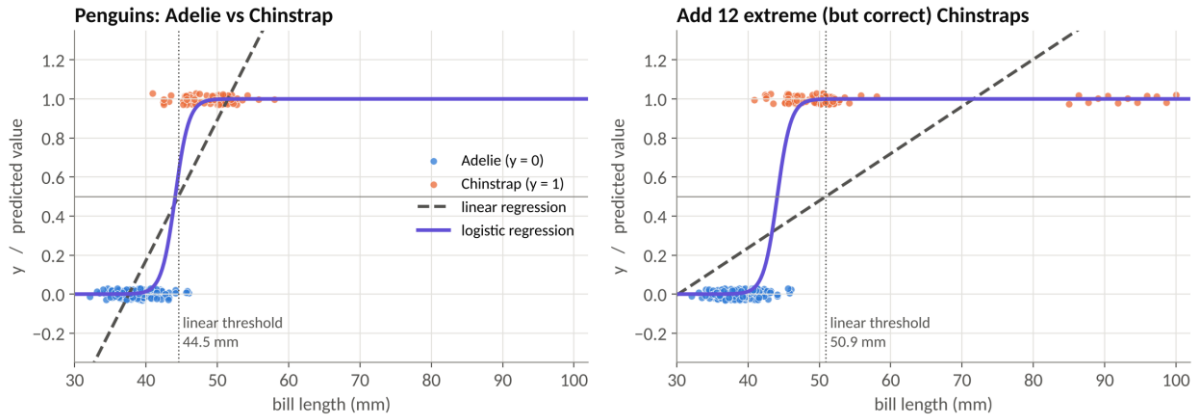


Figure 1. Left: least squares (dashed) and logistic regression (purple) on Adelie vs Chinstrap. Right: the same after adding twelve Chinstraps with extremely long bills.

On the left this seems to work: the line crosses 0.5 at 44.5 mm, a sensible threshold. But there are two serious problems.

1. **The outputs are not probabilities.** The line goes below 0 for short bills and above 1 for long ones. A "probability" of 1.3 is meaningless.
2. **Easy points move the threshold.** On the right we add twelve Chinstraps with very long bills. They are the easiest birds to classify — nothing should change. But squared error punishes their large residuals, so the line tilts to reach them and the threshold moves from 44.5 mm to 50.9 mm. Now many ordinary Chinstraps between 45 and 51 mm are misclassified, because of points that were never in doubt.

The purple S-shaped curve is **logistic regression**. It stays between 0 and 1, and adding the easy points barely changes it. Section 2 builds it.

1.4 Two roads to P(y | x)

There are two fundamentally different ways to obtain the class probabilities, and this week covers both:

Table 2. Discriminative and generative classifiers.

	Discriminative	Generative
What is modelled	P(y | x) directly	P(y) and P(x | y) — how each class produces its features
How P(y | x) is obtained	it is the model's output	Bayes' theorem: P(y | x) ∝ P(x | y) P(y)
This week	logistic regression, softmax regression (Sections 2–4)	Naive Bayes (Section 5)
Can it generate data?	no	yes — it models the features of each class

An analogy: to tell two languages apart, a discriminative learner learns only the differences that matter for telling them apart; a generative learner learns each language well enough to speak it, and then asks which language more probably produced a given sentence. The second is more work, but — if its assumptions are right — it needs fewer examples. We test exactly this claim in Section 5.9.

1.5 The data used in these notes

Table 3. Data sets.

Data set	Examples	Features	Task
Palmer penguins	219 (Adelie vs Chinstrap); 342 (three species)	bill length, bill depth (mm)	binary and 3-class
Wisconsin breast cancer	569 biopsies, 212 malignant	30 measurements of cell nuclei	malignant vs benign
SMS Spam Collection	5,572 messages, 747 spam	word counts (7,387 words)	spam vs ham

COMMON MISTAKE

Always compare with a **baseline**. In the SMS data, 87% of messages are ham, so a "model" that always says "not spam" already has about 87% accuracy (86.58% on our test set). An accuracy of 90% would be almost worthless here.

2. Logistic regression

Despite its name, logistic regression is a **classification** method. It is probably the most widely used classifier in the world: in medicine, finance, advertising, and as the first baseline in almost every machine learning project. The name comes from the **logistic function** (the sigmoid) and from the fact that it is a linear regression on the **log-odds**, as we will see.

History. The logistic curve was introduced by Pierre-François Verhulst (around 1838–1845) to model population growth; Joseph Berkson coined the word "logit" in 1944; David Cox developed logistic regression for binary data in 1958.

2.1 The sigmoid function

DEFINITION — SIGMOID (LOGISTIC) FUNCTION

σ(z) = 1 / (1 + e^(−z)). It maps any real number z to a number strictly between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

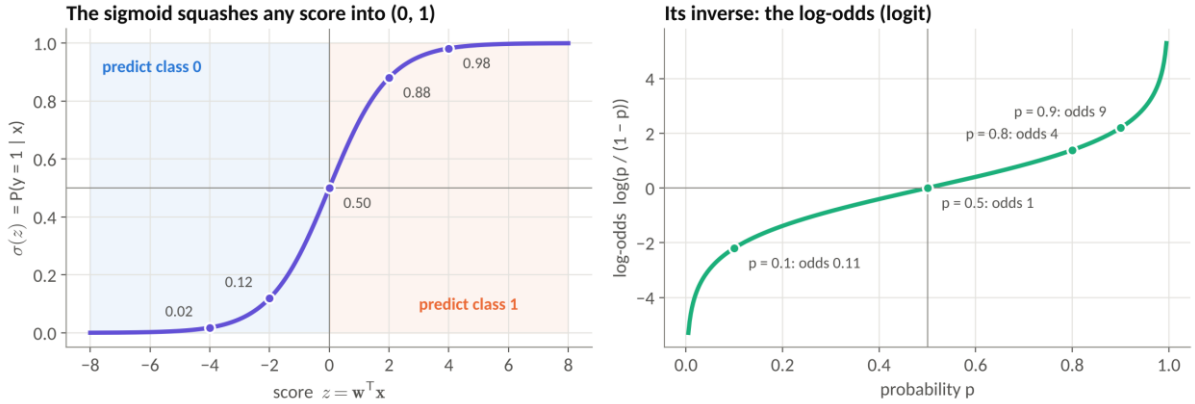


Figure 2. Left: the sigmoid. Right: its inverse, the log-odds (logit).

Important properties, all visible in Figure 2:

- **Range:** 0 < σ(z) < 1 for every z. Large positive z gives values close to 1; large negative z, close to 0.
- **Centre:** σ(0) = 1/(1 + 1) = 0.5.
- **Symmetry:** σ(−z) = 1 − σ(z).
- **Saturation:** σ(2) ≈ 0.88, σ(4) ≈ 0.98, σ(−4) ≈ 0.02. A score of 4 already means "98% sure".
- **Derivative:** σ′(z) = σ(z)(1 − σ(z)) — we use this in Section 3.7. Its largest value is 1/4, at z = 0.

Table 4. Values of the sigmoid worth remembering.

z	−4	−2	−1	0	1	2	4
σ(z)	0.018	0.119	0.269	0.500	0.731	0.881	0.982

2.2 The logistic regression model

Logistic regression combines a linear score — exactly as in linear regression — with the sigmoid:

$$P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^\top \mathbf{x}}}$$

1. **Score:** $z = \mathbf{w}^\top \mathbf{x} = w_0 + w_1 x_1 + \ldots + w_d x_d$. As in Lecture 2 we use the *bias trick*: x has a first entry equal to 1, so w_0 is the intercept. The score can be any real number.
2. **Probability:** $p = \sigma(z) = P(y = 1 \mid x)$. The sigmoid squashes the score into (0, 1).
3. **Decision (only when needed):** $\hat{y} = 1$ if $p \geq t$, usually $t = 0.5$.

With d features the model has **d + 1 parameters**: one weight per feature plus the bias. For the 30 breast-cancer features, that is 31 numbers. Everything you learned about design matrices, feature scaling and regularisation in Lecture 2 carries over unchanged.

2.3 Odds and log-odds: what the score means

Solve the model for the score. If $p = \sigma(z)$, then $z = \log(p/(1-p))$. The ratio $p/(1-p)$ is called the **odds**:

DEFINITION — ODDS AND LOG-ODDS

The **odds** of an event with probability p are $p/(1-p)$. The **log-odds** (or **logit**) are $\log(p/(1-p))$.

$$\log \frac{P(y=1 \mid \mathbf{x})}{1 - P(y=1 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x} = w_0 + w_1 x_1 + \cdots + w_d x_d$$

So logistic regression says: **the log-odds are a linear function of the features**. This is why it is called logistic *regression*: it is a linear regression on the log-odds scale. Odds come from betting: $p = 0.8$ means odds of $0.8/0.2 = 4$, "4 to 1".

Table 5. Probability, odds and log-odds.

Probability p	Odds p/(1 − p)	Log-odds	In words
0.01	0.0101	−4.60	1 in 100
0.10	0.111	−2.20	9 to 1 against
0.50	1	0	even
0.75	3	1.10	3 to 1 on
0.90	9	2.20	9 to 1 on
0.99	99	4.60	99 to 1 on

Notice the three scales: probability runs from 0 to 1, odds from 0 to ∞, and log-odds from −∞ to +∞. Only the log-odds can take any real value, which is why it is the quantity we set equal to the linear score.

2.4 The decision boundary

With threshold 0.5 we predict class 1 exactly when $p \geq 0.5$, which happens exactly when the score is non-negative:

$$P(y = 1 \mid \mathbf{x}) \geq 0.5 \iff \mathbf{w}^{\mathsf{T}}\mathbf{x} \geq 0$$

The set of points where $w^Tx = 0$ is a straight line in two dimensions, a plane in three, and a **hyperplane** in general. Logistic regression is therefore a **linear classifier**: it separates the classes with a hyperplane. Figure 3 shows the penguins with two features.

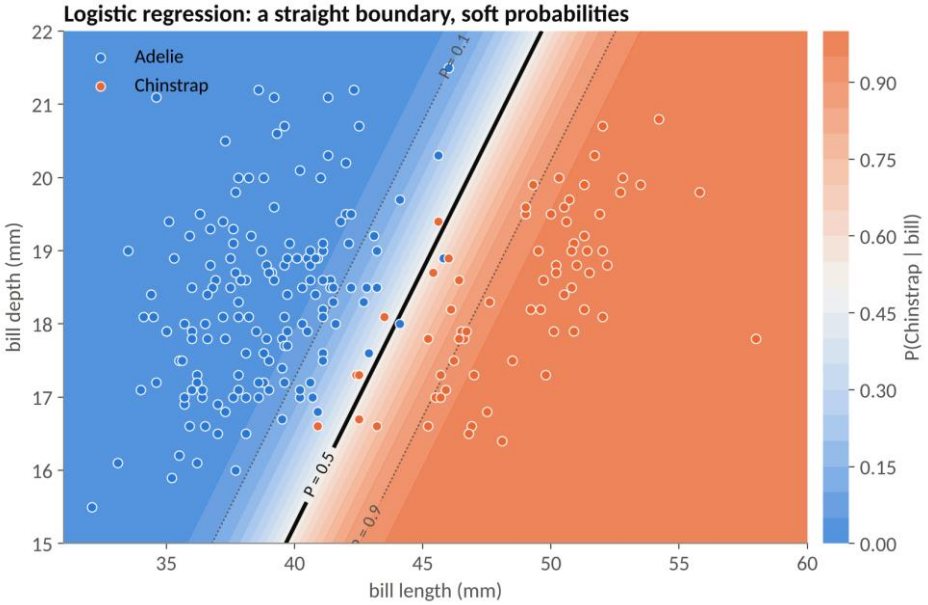


Figure 3. Logistic regression on Adelie (blue) vs Chinstrap (orange). Background: P(Chinstrap); black line: decision boundary; dotted: P = 0.1 and 0.9.

- The background colour is the predicted probability. Near the boundary the model is unsure (white); far from it, confident.
- The 0.1 and 0.9 contours are parallel to the boundary: the probability depends only on the distance from the line.
- With standardised features the weights are about −1.95 (bias), +4.17 (length), −1.29 (depth): longer bills push towards Chinstrap, deeper bills slightly towards Adelie. Training accuracy is 96.8% — a few birds lie on the wrong side because the classes overlap.
- If we multiply all weights by 2, the boundary $w^Tx = 0$ stays exactly where it is, but the probabilities become sharper. The *direction* of w sets the boundary; its *size* sets the confidence.

2.5 Worked example in real units

WORKED EXAMPLE — ONE FEATURE, MILLIMETRES

With bill length as the only feature, the fitted model is: **log-odds(Chinstrap) = -46.258 + 1.049 × length**.

Boundary: set the log-odds to 0: length = 46.258 / 1.049 ≈ **44.09 mm**. Shorter: Adelie; longer: Chinstrap.

40 mm: log-odds = −46.258 + 1.049 × 40 = −4.30; p = σ(−4.30) ≈ **0.013**.

45 mm: log-odds = −46.258 + 1.049 × 45 = 0.95; p = σ(0.95) ≈ **0.722**.

50 mm: log-odds = 6.19; p ≈ **0.998**.

Ten millimetres take us from "almost surely Adelie" to "almost surely Chinstrap". The steepness is set by the slope, 1.049 per millimetre.

2.6 Interpreting coefficients: odds ratios

If feature x_j increases by one unit while all other features stay the same, the log-odds increase by w_j. Exponentiating, the **odds are multiplied by e^(w_j)**:

$$\frac{\text{odds}(x_j + 1)}{\text{odds}(x_j)} = e^{w_j}$$

DEFINITION — ODDS RATIO

e^(w_j) is the factor by which the odds of class 1 are multiplied when x_j increases by one unit, other features held fixed. e^(w_j) > 1: the feature pushes towards class 1; < 1: towards class 0; = 1 (w_j = 0): no effect.

WORKED EXAMPLE — ODDS RATIO OF BILL LENGTH

e^1.049 ≈ **2.855**: each extra millimetre of bill multiplies the odds of being a Chinstrap by about 2.9. Two extra millimetres multiply them by $2.855^2 \approx 8.15$.

Check with the numbers: at 43 mm, log-odds = −1.15, p = 0.24, odds = 0.32; at 45 mm, log-odds = 0.95, p = 0.72, odds = 2.6. Indeed 2.6/0.32 ≈ 8.1. Note that the **odds** were multiplied by 8, but the **probability** only tripled.

COMMON MISTAKE

Odds ratios act on **odds**, not on probabilities. "The odds are multiplied by 2" does not mean "the probability doubles".

Coefficients depend on **units** (per mm ≠ per cm). To compare features, **standardise** them first; the odds ratio is then "per one standard deviation".

Correlated features **share credit**: a coefficient means "given all the other features in the model".

A coefficient describes an **association** in the data, not a cause.

Figure 4 applies this to the breast-cancer data (30 standardised features). Orange bars push towards malignant: "worst texture" has an odds ratio of about 3.9 per standard deviation — large, irregular, unevenly textured nuclei are signs of cancer. Some blue bars, such as "compactness error", belong to features strongly correlated with others already in the model; their negative sign is conditional on those features and must not be read as "protective".

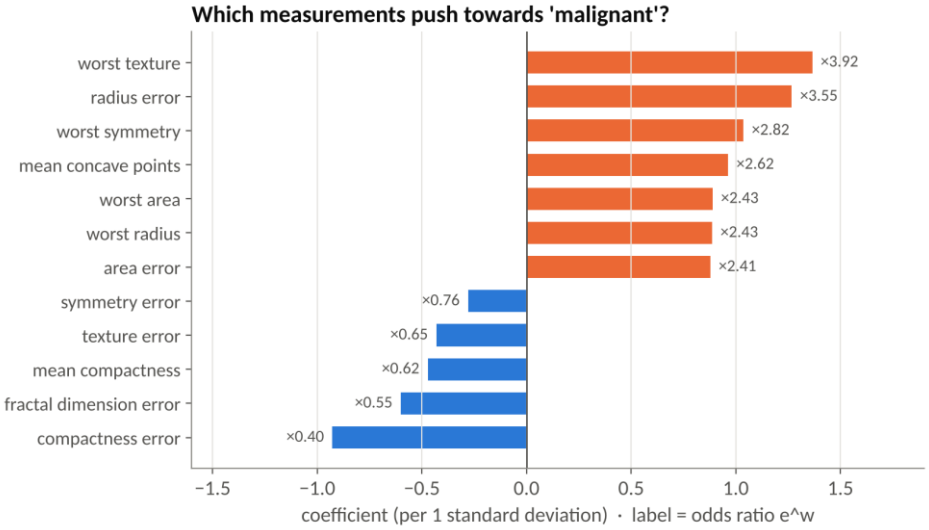


Figure 4. Largest positive and negative coefficients of logistic regression on the breast-cancer data, with odds ratios per standard deviation.

2.7 From probabilities to decisions: choosing the threshold

The default threshold 0.5 is only correct when both kinds of mistakes cost the same. Let C_FP be the cost of a **false positive** (predicting 1 when the truth is 0, e.g. a false alarm) and C_FN the cost of a **false negative** (predicting 0 when the truth is 1, e.g. a missed cancer). If the model says P(y = 1 | x) = p, the expected costs of the two decisions are:

$$\mathrm{E}[\text{cost of predicting } 1] = (1-p)\,C_{FP} \qquad \mathrm{E}[\text{cost of predicting } 0] = p\,C_{FN}$$

We should predict 1 when that is the cheaper decision:

$$\text{predict } 1 \iff p\,C_{FN} \geq (1-p)\,C_{FP} \iff p \geq t^{*} = \frac{C_{FP}}{C_{FP}+C_{FN}}$$

KEY IDEA

The optimal threshold is t = C_FP / (C_FP + C_FN). If a missed cancer is 20 times worse than a false alarm, t = 1/21 ≈ 0.05 — far below 0.5. The model does not change; only the decision rule does.

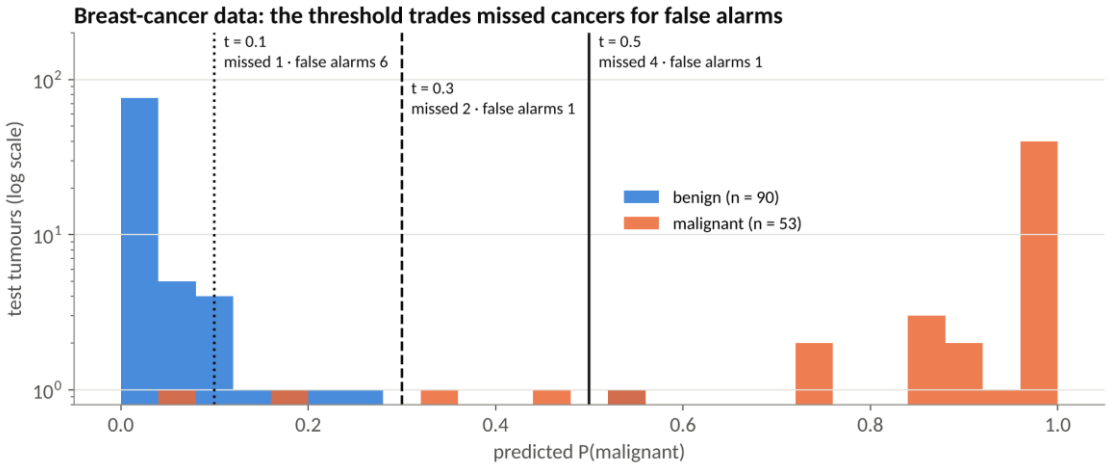


Figure 5. Predicted P(malignant) for the 143 test biopsies and three thresholds.

Table 6. Effect of the threshold on the breast-cancer test set.

Threshold t	Missed cancers (FN)	False alarms (FP)
0.5	4	1
0.3	2	1
0.1	1	6

COMMON MISTAKE

Never choose the threshold by looking at the **test** set — that makes your reported results optimistic. Choose it on a validation set (or by cross-validation) and evaluate on the test set once.

2.8 Counting mistakes: the confusion matrix

DEFINITION — CONFUSION MATRIX

A table that counts every combination of true and predicted class. Rows = truth, columns = prediction. For two classes: true negatives (TN), false positives (FP), false negatives (FN) and true positives (TP).

Table 7. Layout of a binary confusion matrix.

	predicted 0	predicted 1
true 0	TN (correct)	FP (false alarm)

	predicted 0	predicted 1
true 1	FN (missed)	TP (correct)

$$\text{accuracy} = \frac{TP + TN}{n}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad \text{precision} = \frac{TP}{TP + FP}$$

- **Accuracy:** fraction of all predictions that are correct.
- **Recall** (sensitivity, true positive rate): of all real positives, the fraction we found.
- **Precision:** of all cases we called positive, the fraction that really were positive.

WORKED EXAMPLE — BREAST CANCER AT T = 0.5

Confusion matrix: TN = 89, FP = 1, FN = 4, TP = 49.

Accuracy = (89 + 49) / 143 = **96.5%**.

Recall = 49 / (49 + 4) = **92.5%** of cancers found.

Precision = 49 / (49 + 1) = **98.0%** of alarms were real.

Lowering the threshold moves cases from "predicted benign" to "predicted malignant": FN goes down (recall up), FP goes up (precision usually down).

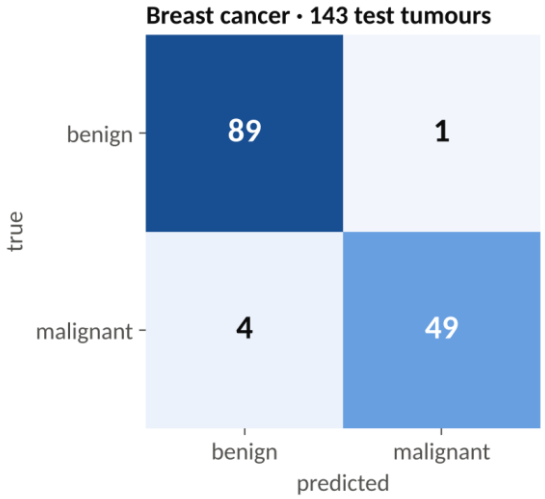


Figure 6. Confusion matrix of logistic regression on the breast-cancer test set.

Precision–recall trade-offs and ROC curves are studied in depth later in the course, with model evaluation.

TRY IT YOURSELF

A standardised feature has weight w = −0.69. What happens to the odds when the feature rises by one standard deviation?

3. Maximum likelihood estimation

We now have a model, $P(y = 1 \mid x) = \sigma(w^T x)$, but we have not said how to choose w. Empirical risk minimisation says: choose a loss and minimise the average loss on the training data. But which loss? In Lecture 2 we simply chose squared error. This section derives the loss from one of the most important principles in statistics and machine learning: **maximum likelihood**. Ronald Fisher developed and popularised it between 1912 and 1922; today it is the training objective of most statistical models and of most neural networks, including large language models.

3.1 The principle

DEFINITION — MAXIMUM LIKELIHOOD

Choose the parameters under which the observed data would have been **most probable**. The **likelihood** $L(\theta)$ is the probability (or density) of the observed data, viewed as a function of the parameters θ.

Think of a detective with several suspects. Each suspect would explain the evidence with a different probability. The detective chooses the suspect under whom the evidence is least surprising. Our "suspects" are the possible parameter values; the "evidence" is the training data.

The recipe, which we apply four times in these notes:

1. Write a **probability model for one example**: $P(y_i \mid x_i; \theta)$.
2. **Multiply** over the examples (assuming they are independent): $L(\theta) = \Pi_i P(y_i \mid x_i; \theta)$.
3. **Take the log**: products become sums.
4. **Maximise** — or, equivalently, **minimise the negative log-likelihood (NLL)**. The NLL is the loss.

$$\hat{\mathbf{w}}_{\text{MLE}} = \arg\max_{\mathbf{w}} L(\mathbf{w}) = \arg\min_{\mathbf{w}} \left[-\sum_{i=1}^{n} \log P(y_i \mid \mathbf{x}_i; \mathbf{w}) \right]$$

3.2 A first example: a biased coin

A coin lands heads with an unknown probability p. We flip it 10 times and observe 7 heads and 3 tails. What is our best estimate of p?

WORKED EXAMPLE — MAXIMUM LIKELIHOOD FOR A COIN

Step 1–2. If the bias were p, the probability of our exact sequence would be $\mathbf{L(p) = p^7(1 - p)^3}$.

Try some values: $L(0.5) \approx 0.00098$, $L(0.7) \approx 0.00222$, $L(0.9) \approx 0.00048$. The data are most probable near $p = 0.7$.

Step 3. $\log L(p) = 7 \log p + 3 \log(1 - p)$.

Step 4. Differentiate and set to zero: $7/p - 3/(1 - p) = 0 \Rightarrow 7(1 - p) = 3p \Rightarrow \mathbf{\hat{p} = 0.7}$.

In general, the maximum-likelihood estimate of a coin's bias is the **fraction of heads**: $\hat{p}$ = (number of heads)/n.

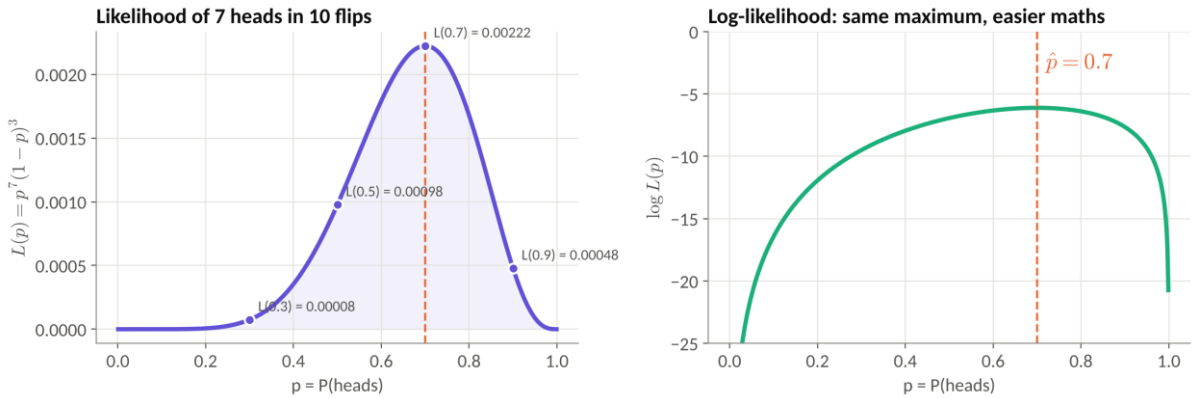


Figure 7. Likelihood (left) and log-likelihood (right) of 7 heads in 10 flips. Both are maximised at p = 0.7.

COMMON MISTAKE

L(0.7) = 0.0022 is **not** the probability that p = 0.7. It is the probability of the **data** if p were 0.7. The likelihood is a function of the parameter, not a probability distribution over it (it does not even integrate to 1).

3.3 Why we always work with the log

- **Same maximum.** The logarithm is strictly increasing, so the parameter that maximises log L also maximises L.
- **Products become sums.** A sum over examples is exactly the average loss that ERM needs, and sums are much easier to differentiate than products.
- **No underflow.** If each of 1,000 examples has probability 0.5, the likelihood is $0.5^{1000} \approx 9.3 \times 10^{-302}$. With about 1,075 such examples, the product is smaller than the smallest positive 64-bit floating-point number and the computer stores exactly 0. The log-likelihood, $1000 \times \log 0.5 \approx -693$, is a perfectly ordinary number.

```
0.5 ** 1000          # 9.33e-302
0.5 ** 1100          # 0.0      <- underflow: the likelihood is 'zero'
1000 * np.log(0.5)   # -693.1   <- the log-likelihood is fine
```

3.4 Least squares is maximum likelihood

At the end of Lecture 2 we claimed that least squares is maximum likelihood under Gaussian noise. Now we can prove it. Assume $y_i = w^T x_i + \varepsilon_i$, where the noise ε_i is normally distributed with mean 0 and variance σ^2, independently for each example.

1. One example: the density of y_i is $(1/\sqrt{(2\pi\sigma^2)}) \exp(-(y_i - w^T x_i)^2/(2\sigma^2))$.
2. Its negative log: $\tfrac{1}{2} \log(2\pi\sigma^2) + (y_i - w^T x_i)^2/(2\sigma^2)$.
3. Sum over examples. The first term does not depend on w, so it is a constant:

$$y_i \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i,\ \sigma^2) \ \Rightarrow\ -\log L(\mathbf{w}) = \frac{1}{2\sigma^2} \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \text{const}$$

Minimising the negative log-likelihood is therefore exactly minimising the sum of squared residuals. Squared error is not an arbitrary choice: it is what maximum likelihood gives if you believe the noise is

Gaussian. This also tells us when squared error is a *bad* choice — when the noise is not Gaussian. Laplace-distributed noise gives the absolute error; and 0/1 labels need a completely different noise model.

KEY IDEA

Choosing a loss function is choosing a noise model. Gaussian → squared error; Laplace → absolute error; Bernoulli (coin-flip) labels → log loss.

3.5 The likelihood of logistic regression and the log loss

For a 0/1 label the right model is a weighted coin — a **Bernoulli** distribution. Logistic regression says: for input x, the label is 1 with probability $p = \sigma(w^T x)$. A compact way to write "p if y = 1, and 1 − p if y = 0" in one formula is:

$$P(y \mid \mathbf{x}; \mathbf{w}) = p^{\,y}\,(1-p)^{1-y}, \qquad p = \sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x})$$

Check: if y = 1, the second factor has exponent 0 and we get p; if y = 0, the first factor disappears and we get 1 − p. Each example has its own coin, because each has its own x_i and therefore its own p_i. Multiplying over examples (step 2):

$$L(\mathbf{w}) = \prod_{i=1}^{n} P(y_i \mid \mathbf{x}_i; \mathbf{w}) = \prod_{i=1}^{n} p_i^{\,y_i}(1-p_i)^{1-y_i}$$

Take the log (step 3) — the exponents come down — and put a minus sign in front and divide by n (step 4):

$$J(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}\left[\, y_i \log p_i + (1-y_i)\log(1-p_i)\,\right]$$

DEFINITION — LOG LOSS (BINARY CROSS-ENTROPY)

$J(w) = -(1/n)\,\Sigma_i\,[y_i \log p_i + (1 - y_i)\log(1 - p_i)]$. For one example it equals −log(probability given to the true class). It is also called **binary cross-entropy** or the **negative log-likelihood**; in PyTorch it is BCELoss.

Table 8. Log loss for one example.

P(true class)	Loss −log p	Verdict
0.99	0.01	confident and right
0.90	0.11	right
0.50	0.69 (= log 2)	doesn't know
0.10	2.30	wrong
0.01	4.61	confidently wrong
→ 0	→ ∞	certain and wrong

The log loss is gentle with uncertainty but brutal with **confident mistakes**. A model that predicts p = 0.5 for everything has log loss log 2 ≈ 0.693 on any data set; this is the starting loss of gradient descent from w = 0.

3.6 Why not squared error on the probabilities?

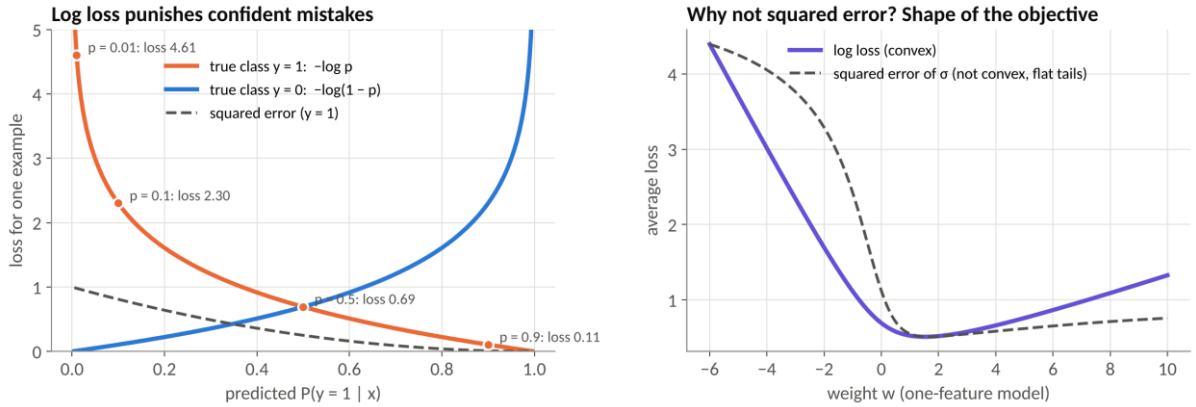


Figure 8. Left: per-example losses; squared error (dashed) hardly punishes confident mistakes. Right: for a one-weight model, log loss is convex while squared error of the sigmoid has flat tails.

- **Confident mistakes.** With y = 1 and p = 0.01, squared error is 0.98 — barely more than for p = 0.5. The log loss is 4.6 and grows without bound.
- **Vanishing gradients.** The sigmoid is flat in its tails, so the gradient of the squared error is tiny exactly for badly wrong predictions: a wrong model learns very slowly.
- **Convexity.** For logistic regression the log loss is convex — one bowl, and gradient descent always finds the minimum. Squared error of the sigmoid is not convex: it has flat plateaus where gradient descent stalls.

3.7 Deriving the gradient

We need the gradient of J(w) for gradient descent. First, a useful property of the sigmoid:

$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

(Differentiate σ(z) = (1 + e^(−z))^(−1) to get e^(−z)/(1 + e^(−z))², which equals σ(z)(1 − σ(z)).) Now use the chain rule for one example, ℓ = −y log p − (1 − y) log(1 − p), with p = σ(z) and z = $\mathbf{w}^{\mathsf{T}}\mathbf{x}$:

$$\frac{\partial \ell_i}{\partial \mathbf{w}} = \frac{\partial \ell_i}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \mathbf{w}} = \left(-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i} \right) p_i (1 - p_i)\, \mathbf{x}_i = (p_i - y_i)\, \mathbf{x}_i$$

1. ∂ℓ/∂p = −y/p + (1 − y)/(1 − p).
2. ∂p/∂z = σ′(z) = p(1 − p).
3. ∂z/∂w = x.
4. Multiply the first two: (−y/p + (1 − y)/(1 − p)) · p(1 − p) = −y(1 − p) + (1 − y)p = **p − y**.
5. So ∂ℓ/∂w = (p − y) x: **prediction error times input**.

Averaging over all examples and writing it with the design matrix X:

$$\nabla J(\mathbf{w}) = \frac{1}{n} X^\top (\sigma(X\mathbf{w}) - \mathbf{y})$$

KEY IDEA

Compare with linear regression (Lecture 2): ∇J = (2/n) X^T(Xw − y). Both have the form **X^T × (prediction − truth)**. This is not a coincidence: it is what you get whenever the loss is the negative log-likelihood of the right distribution for the output. You will see the same pattern a third time with softmax (Section 4.3) and again in the output layer of neural networks.

An example the model predicts perfectly (p = y) contributes nothing to the gradient; a badly wrong one contributes almost its full input vector.

Unlike linear regression, setting this gradient to zero gives a non-linear equation in w, so there is **no closed-form solution** (no "normal equations"). We must use an iterative method such as gradient descent.

3.8 Gradient descent for logistic regression

The algorithm is last week's, with one change: the predictions go through the sigmoid.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{1}{n} X^\top (\sigma(X\mathbf{w}) - \mathbf{y})$$

```
def sigmoid(z):
    z = np.clip(z, -500, 500)          # numerically stable: no overflow in exp
    return 1 / (1 + np.exp(-z))

Xs = (X - X.mean(0)) / X.std(0)        # standardise (training statistics)
A = np.c_[np.ones(n), Xs]              # bias column
w = np.zeros(A.shape[1]); eta = 0.5
for step in range(300):
    p = sigmoid(A @ w)                 # probabilities
    grad = A.T @ (p - y) / n           # (1/n) X^T (sigma(Xw) - y)
    w = w - eta * grad
```

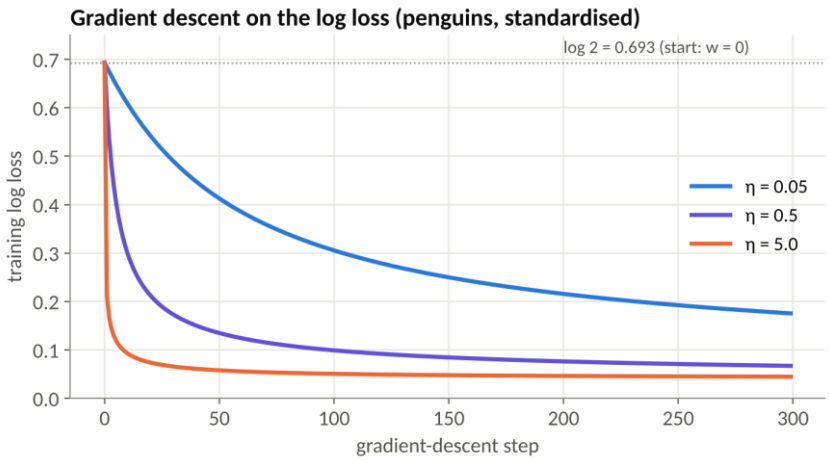


Figure 9. Training log loss for three learning rates on the penguins (standardised features). All start at log 2.

Two practical remarks. (1) Logistic regression tolerates much larger learning rates than linear regression: its curvature is bounded because σ′(z) ≤ 1/4. Here even η = 5 converges. (2) For very negative scores, np.exp(−z) can overflow; clipping z (or a two-branch formula) avoids this.

3.9 Perfectly separable data and regularisation

If a straight line separates the two classes perfectly, maximum likelihood has no finite solution. With a perfect separating line we can always make the training probabilities even closer to 0 and 1 by making w larger: the boundary stays the same, the sigmoid becomes steeper, and the loss gets closer to zero without ever reaching it. The weights grow forever and the model becomes absurdly overconfident.

Table 9. Adelie vs Gentoo (perfectly separable, standardised bill length and depth): the weights grow as regularisation is weakened.

scikit-learn C	Penalty λ = 1/C	Largest |w|	Training accuracy
1 (default)	1	2.9	99.6%
100	0.01	8.7	100%
10^4	10^{-4}	24.4	100%
10^6	10^{-6}	43.3	100%
10^8	10^{-8}	63.4	100%

The cure is the L2 penalty from Lecture 2:

$$J_\lambda(\mathbf{w}) = J(\mathbf{w}) + \frac{\lambda}{2}\,\|\mathbf{w}\|_2^2 \qquad (\text{scikit-learn}:\ C = 1/\lambda)$$

COMMON MISTAKE

In scikit-learn, LogisticRegression is **regularised by default** with C = 1, and C is the **inverse** of the penalty strength: larger C means less regularisation. (This is the opposite of Ridge's alpha — a classic exam trap.) It is also why scikit-learn's weights differ slightly from unregularised code you write yourself.

3.10 Regularisation is a prior: MLE and MAP

Maximum likelihood uses only the data. Often we also have prior beliefs about the parameters — "weights are probably small", or "a coin is probably roughly fair". Bayes' theorem combines the two: posterior ∝ likelihood × prior. Choosing the parameters that maximise the posterior is **MAP (maximum a posteriori)** estimation:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta}\ P(\text{data} \mid \theta)\, P(\theta)$$

Taking minus the log: −log posterior = (negative log-likelihood) + (−log prior) = **loss + penalty**. A Gaussian prior centred at zero gives exactly the L2 penalty (ridge, regularised logistic regression); a Laplace prior gives the L1 penalty (lasso). For counts, a prior that adds imaginary observations gives **Laplace smoothing** (Section 5.7).

WORKED EXAMPLE — A PRIOR FOR A COIN

3 heads in 3 flips: MLE gives $\hat{p} = 3/3 = 1$, i.e. "tails are impossible" — overconfident after three flips.

Add one imaginary head and one imaginary tail (a simple prior towards 0.5): $\hat{p} = (3 + 1)/(3 + 2) = 0.8$. Much more reasonable.

TRY IT YOURSELF

Compute the log loss of each prediction: (a) a spam message with P(spam) = 0.9; (b) a ham message with P(spam) = 0.4; (c) a ham message with P(spam) = 0.99.

4. Multiclass classification

Many problems have more than two classes: three penguin species, ten digits, thousands of object categories. There are two strategies: combine several binary classifiers, or build one model with K outputs.

4.1 Combining binary classifiers

- **One-vs-rest (one-vs-all):** train K binary classifiers, "class k versus all the others". Predict the class whose classifier is most confident: $\hat{y} = \arg\max_k \sigma(w_k^T x)$.
- **One-vs-one:** train a classifier for every pair of classes, $K(K-1)/2$ in total, and let them vote.

Table 10. Number of classifiers needed.

Classes K	One-vs-rest	One-vs-one
3 (penguins)	3	3
10 (digits)	10	45
1,000 (images)	1,000	499,500

Weak points of one-vs-rest: the K scores are trained separately and do **not** sum to 1, so they are not a proper probability distribution; the "rest" class is often much larger than the "one" class; and where the K boundaries cross there are ambiguous regions (Figure 11, left).

4.2 The softmax function

Give each class k its own weight vector w_k and compute one score per class, $z_k = w_k^T x$. The **softmax** turns K arbitrary scores into K probabilities: exponentiate (making them positive) and normalise (making them sum to 1).

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^\top \mathbf{x}}}$$

WORKED EXAMPLE — SOFTMAX BY HAND

Scores for (Adelie, Chinstrap, Gentoo): z = (2, 1, 0.1).

Exponentials: $e^2 \approx 7.39$, $e^1 \approx 2.72$, e^0.1 ≈ 1.11; sum ≈ 11.21.

Probabilities: 7.39/11.21 ≈ **0.659**, 2.72/11.21 ≈ **0.242**, 1.11/11.21 ≈ **0.099**. They sum to 1; the largest score gets the largest probability, but the others are not zero.

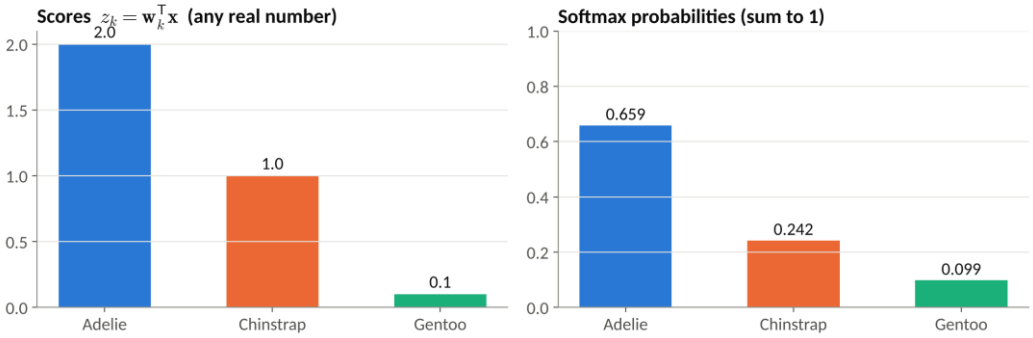


Figure 10. Scores (left) and the softmax probabilities (right) for the example above.

Properties of softmax:

- **Shift invariance:** adding the same constant c to every score changes nothing, because e^c cancels in the numerator and denominator. This is used for numerical stability (Section 4.6).
- **"Soft" maximum:** a hard maximum would give (1, 0, 0); softmax is a smooth, differentiable version.
- **Two classes give the sigmoid:**

$$\frac{e^{z_1}}{e^{z_0}+e^{z_1}} = \frac{1}{1+e^{-(z_1-z_0)}} = \sigma(z_1 - z_0)$$

So logistic regression is exactly softmax regression with K = 2. Softmax is also the last layer of almost every neural-network classifier and the way a language model turns scores into probabilities over its vocabulary.

4.3 Softmax regression and the cross-entropy loss

Softmax regression (also called **multinomial logistic regression**) has a weight matrix W = [w_1 … w_K] with (d + 1) × K numbers, and predicts P = softmax(XW). To train it we follow the maximum-likelihood recipe again, now with a K-sided die (a **categorical** distribution) instead of a coin. The probability of the observed class of example i is p_{i, y_i}; the negative log-likelihood, averaged, is the **cross-entropy**:

$$J(W) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K}\mathbf{1}[y_i = k]\,\log p_{ik}$$

The indicator $\mathbf{1}[y_i = k]$ simply picks out the probability of the true class: again, the loss is −log(probability given to the truth). To write things with matrices we encode labels as **one-hot** vectors:

Species	Class number	One-hot vector
Adelie	0	[1, 0, 0]
Chinstrap	1	[0, 1, 0]
Gentoo	2	[0, 0, 1]

Stacking the one-hot rows gives an n × K matrix Y, and the predicted probabilities an n × K matrix P. The gradient has — for the third time — the form "input-transpose times (prediction − truth)":

$$\nabla_W J = \frac{1}{n} X^\top (P - Y)$$

Gradient-descent code for softmax regression is the logistic-regression code with sigmoid replaced by softmax and the vector y replaced by the one-hot matrix Y. Starting from W = 0, every class gets probability 1/K, so the initial loss is log K (log 3 ≈ 1.10 for the penguins).

4.4 Example: three penguin species

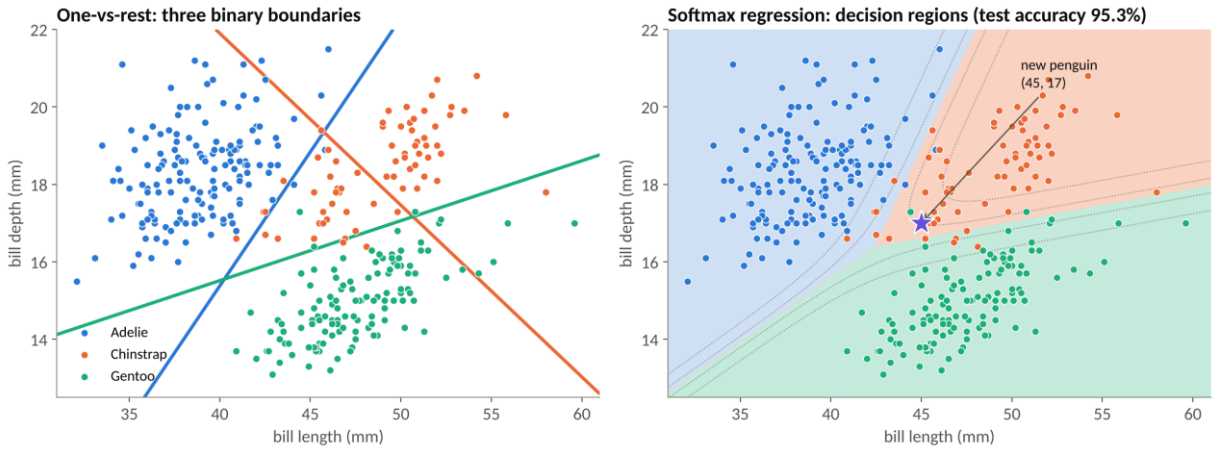


Figure 11. Left: one-vs-rest boundaries cross and leave ambiguous regions. Right: softmax regression divides the plane into three regions; the star is a new penguin.

- Softmax regression is still a **linear classifier**: the boundary between classes j and k is where $z_j = z_k$, i.e. $(w_j - w_k)^T x = 0$ — a straight line.
- Test accuracy: **95.3%** on 86 held-out penguins, with only two measurements.
- A new penguin with a 45 mm long, 17 mm deep bill lies near the junction of the three regions. The model gives Adelie 0.163, Chinstrap 0.549, Gentoo 0.288: it predicts Chinstrap, but honestly says it is only about 55% sure.

4.5 Evaluating a multiclass model

The confusion matrix becomes K × K; the diagonal holds the correct predictions.

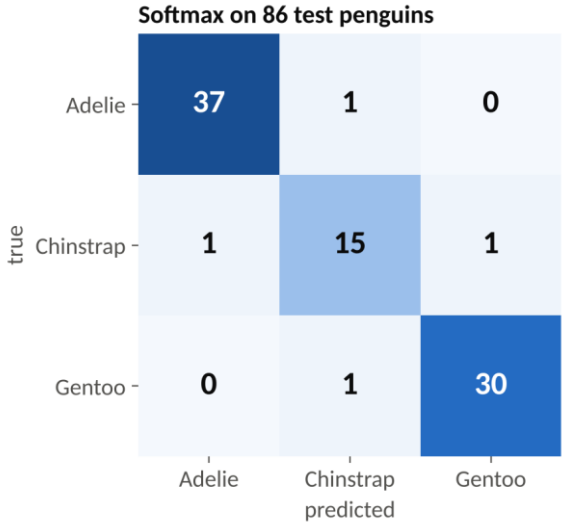


Figure 12. Confusion matrix of softmax regression on the penguin test set.

- Overall accuracy: 82 of 86 = 95.3%.
- All four errors involve **Chinstrap**, which lies between the other two species in bill space.
- **Per-class recall** for Chinstrap is 15/17 ≈ 88%, lower than the overall accuracy. The smallest class often suffers most — always look per class.
- **Baseline:** always predicting the most common class (Adelie) would score 38/86 ≈ 44%.

4.6 Computing softmax safely

With scores (1000, 1001), np.exp(1000) is infinity and the division gives NaN ("not a number"). Using shift invariance, we subtract each row's largest score before exponentiating: the result is identical, and the largest exponential is now $e^0 = 1$, so nothing can overflow.

```
def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)     # shift: largest score becomes 0
    E = np.exp(Z)                            # every value in (0, 1]
    return E / E.sum(axis=1, keepdims=True)

softmax(np.array([[1000., 1001.]]))          # [[0.269, 0.731]]
```

For the loss, libraries compute log-softmax directly with the "log-sum-exp" trick rather than taking the log of a probability that may already have underflowed to 0. This is why PyTorch's CrossEntropyLoss expects raw scores ("logits"), not probabilities.

TRY IT YOURSELF

With 4 features plus a bias and 5 classes, how many parameters does softmax regression have? How many classifiers would one-vs-one need?

5. Naive Bayes

Naive Bayes takes the **generative** road. Instead of learning P(y | x) directly, it models how each class generates its features, and then uses Bayes' theorem to turn this around. It is one of the oldest machine-learning methods, it trains in a single pass over the data — just counting and averaging — and on text it is still a strong baseline.

5.1 Bayes' theorem

The joint probability of x and y can be written in two ways:

$$P(\mathbf{x}, y) = P(\mathbf{x} \mid y)\, P(y) = P(y \mid \mathbf{x})\, P(\mathbf{x})$$

Dividing by P(x) gives **Bayes' theorem**:

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y)\, P(y)}{P(\mathbf{x})}$$

Table 11. The four parts of Bayes' theorem.

Term	Name	Meaning
P(y)	prior	belief about the class before seeing the features (e.g. 13% of SMS are spam)
P(x | y)	likelihood	how typical these features are for class y
P(x)	evidence	how common these features are overall; the same for every class
P(y | x)	posterior	updated belief after seeing the features — what we want

The evidence can be computed from the other parts, $P(x) = \Sigma_k\, P(x \mid y = k)\, P(y = k)$, but for classification we often do not need it: it is the same for every class, so the class with the largest **numerator** P(x | y) P(y) also has the largest posterior. In words: **posterior ∝ likelihood × prior**. Learning from evidence means updating a prior.

History. Thomas Bayes' essay was published in 1763, after his death, by his friend Richard Price; Pierre-Simon Laplace developed the same ideas independently and much further.

5.2 A famous puzzle: the medical test

A disease affects 1% of the people who are screened. The test detects the disease in 90% of ill people (sensitivity), but is also positive for 9% of healthy people (false-positive rate). You test positive. What is the probability that you are ill?

Most people answer about 90%. In a well-known study (Gigerenzer et al., 2007), most of the gynaecologists given a question with exactly these numbers also chose answers far too high. The easiest way to get it right is to **count people**:

Table 12. 1,000 people screened.

	Test positive	Test negative	Total
Ill (1%)	9	1	10

	Test positive	Test negative	Total
Healthy (99%)	89	901	990
Total	98	902	1,000

Of the 98 people who test positive, only 9 are ill: P(ill | positive) = 9/98 ≈ **9.2%**. Bayes' theorem gives the same number:

$$P(\text{ill} \mid \text{positive}) = \frac{0.90 \times 0.01}{0.90 \times 0.01 + 0.09 \times 0.99} = \frac{0.009}{0.0981} \approx 0.092$$

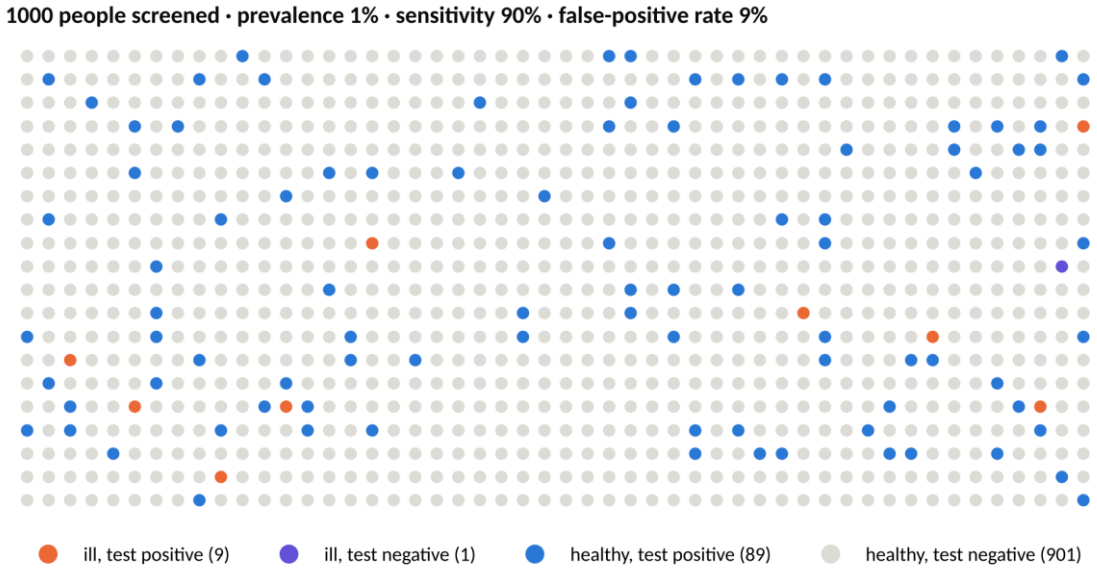


Figure 13. The same 1,000 people as an icon array: the 89 blue false positives far outnumber the 9 orange true positives.

KEY IDEA

Base-rate neglect: when a condition is rare, even a good test produces mostly false alarms, because the healthy group is so much larger. Every classifier for a rare class (fraud, rare diseases, spam in some settings) must take the prior into account — Naive Bayes does so explicitly.

The test is still useful: it raised your probability from 1% to 9%. That is why screening is followed by a second, more specific test.

TRY IT YOURSELF

Repeat the calculation for a prevalence of 10% instead of 1% (same test).

5.3 Bayes as a classifier — and the problem with P(x | y)

A **Bayes classifier** predicts $\hat{y} = \arg\max_k P(x \mid y = k)\, P(y = k)$. If the true distributions were known, this would be the best possible classifier (the **Bayes-optimal** classifier). The prior is easy to estimate: count the classes. The difficulty is the likelihood of a whole feature vector. For a text message, x records which of the 7,387 vocabulary words occur. Even with just "present or absent" there are 2^7387 possible patterns — a number with about 2,224 digits — to estimate from about 4,000 training messages. Impossible. Even 30 yes/no features have $2^{30} \approx 1$ billion combinations. This is the **curse of dimensionality**.

5.4 The naive assumption

DEFINITION — NAIVE BAYES ASSUMPTION

Given the class, the features are **independent** of each other: $P(x_1, \ldots, x_d \mid y) = P(x_1 \mid y) \cdot P(x_2 \mid y) \cdots P(x_d \mid y)$.

$$P(\mathbf{x} \mid y) = P(x_1, \ldots, x_d \mid y) \approx \prod_{j=1}^{d} P(x_j \mid y)$$

Instead of 2^7387 probabilities, we now need one small distribution per feature and class — about 15,000 numbers for the spam filter, each estimated by counting. To avoid underflow we compute in **log space**:

$$\hat{y} = \arg\max_{k} \left[\log P(y = k) + \sum_{j=1}^{d} \log P(x_j \mid y = k) \right]$$

The assumption is almost never true: in spam, "free" and "prize" occur together much more often than independence predicts; in penguins, bill length and depth are correlated within each species. That is why the method is called **naive**. It works surprisingly well anyway, because classification only needs the **right winner**, not the right probability. Naive Bayes double-counts correlated evidence, which makes its probabilities **overconfident** (values like 0.99999 are common), but the ranking of the classes is often still correct (Domingos & Pazzani, 1997).

5.5 Gaussian Naive Bayes

For continuous features we assume, for each class k and feature j, a normal distribution with its own mean μ_{jk} and variance σ^2_{jk}, estimated from the training examples of that class:

$$P(x_j \mid y = k) = \frac{1}{\sqrt{2\pi\sigma^2_{jk}}} \exp\left(-\frac{(x_j - \mu_{jk})^2}{2\sigma^2_{jk}} \right)$$

Training consists of computing the class fractions (priors), and the mean and variance of each feature within each class. That is all.

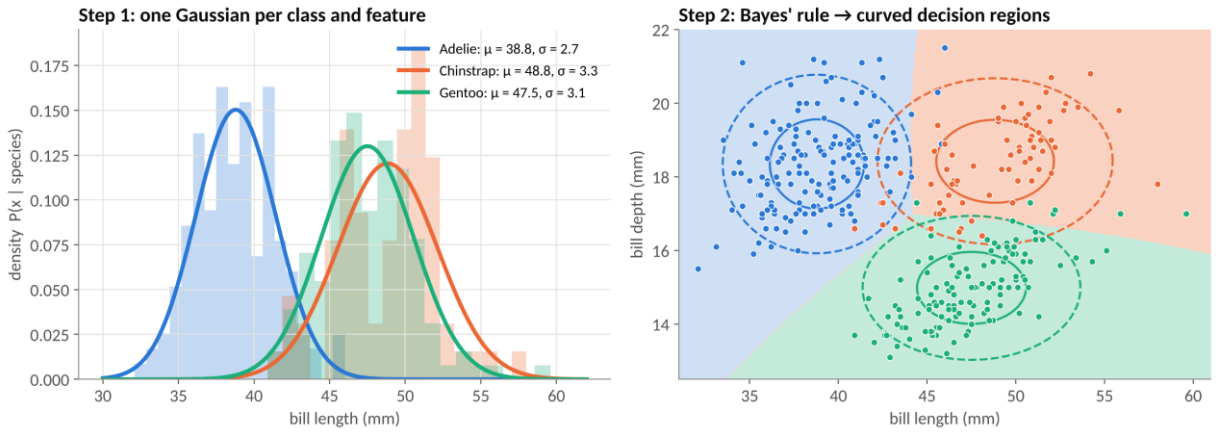


Figure 14. Gaussian Naive Bayes on the penguins. Left: one Gaussian per species for bill length. Right: the resulting decision regions with 1- and 2-standard-deviation ellipses.

- Estimated priors: Adelie 0.441, Chinstrap 0.199, Gentoo 0.359; means (length, depth): Adelie (39.01, 18.47), Chinstrap (48.99, 18.44), Gentoo (47.44, 14.91) mm.
- The ellipses are **axis-aligned**: they cannot tilt along the cloud of points. This is the independence assumption made visible.
- The decision boundaries are **curved**: when the classes have different variances, the log-ratio of two Gaussians is quadratic in x. Gaussian Naive Bayes is not a linear classifier in general.
- Test accuracy: **93.0%**, close to softmax regression's 95.3% — from a model trained by computing a handful of means and variances. No feature scaling is needed.

5.6 Text as numbers: the bag of words

To classify text we must turn it into numbers. The simplest representation is the **bag of words**: build a vocabulary of all words in the training messages, and represent each message by how many times each word occurs. Word order is discarded: "dog bites man" and "man bites dog" get the same vector.

WORKED EXAMPLE — BAG OF WORDS

Message: “Free prize! Call now to claim your free prize”.

Tokens (lower case, punctuation removed): free, prize, call, now, to, claim, your, free, prize.

Count vector: free 2, prize 2, call 1, now 1, to 1, claim 1, your 1, and 0 for the other 7,380 words of the vocabulary.

Almost all entries are zero: the vector is **sparse**, and scikit-learn stores such data as a sparse matrix that keeps only the non-zero counts.

In scikit-learn, CountVectorizer builds the vocabulary (lower-casing and keeping tokens of at least two letters or digits) and the count matrix. It must be fitted on the **training** messages only.

5.7 Multinomial Naive Bayes and Laplace smoothing

Multinomial Naive Bayes imagines that a message of class c is produced by drawing words one at a time from a class-specific "bag of words". The probability of word w in class c is estimated from the training counts: how often w occurs in the messages of class c, divided by the total number of words in that class.

But this raw estimate has a fatal flaw. In our training data the word **"claim"** occurs 85 times in spam and **never** in ham, so the raw estimate of P(claim | ham) is exactly 0. A friend writes *"Did you claim it? Call me when you get home."* Every other word leans towards ham, but one factor of the product P(message | ham) is zero, so the whole product is zero and the model is 100% certain the message is spam. A single unseen word vetoes all other evidence; in log space, log 0 = −∞.

The fix is **Laplace (add-one) smoothing**: pretend that every word was seen α more times in every class (usually α = 1):

$$\hat{P}(w \mid c) = \frac{\text{count}(w, c) + \alpha}{\sum_{v \in V} \text{count}(v, c) + \alpha\,|V|}$$

Adding α|V| to the denominator (|V| = vocabulary size) keeps the probabilities summing to 1. Now P(claim | ham) = (0 + 1)/(46,991 + 7,387) ≈ 1.8×10^{-5}: small, but not zero, and the other words can win. With smoothing, the friendly message gets P(spam) ≈ 0.0001. Smoothing is a **prior** (Section 3.10): imaginary extra observations. In scikit-learn, MultinomialNB(alpha=1.0) is the default.

Table 13. Word counts in the training data (13,172 spam words, 46,991 ham words, |V| = 7,387).

Word	In spam	In ham	P(w | ham), raw	P(w | ham), α = 1
claim	85	0	0	1.8×10^{-5}
prize	63	0	0	1.8×10^{-5}
lunch	0	33	7.0×10^{-4}	6.3×10^{-4}
free	171	43	9.2×10^{-4}	8.1×10^{-4}

Smoothing changes the probabilities of frequent words very little (compare "free") and matters most for rare words. α is a hyperparameter, and should be chosen on validation data.

5.8 A spam filter on real data

The log-odds form of Naive Bayes makes it easy to see how a decision is reached. For two classes:

$$\log \frac{P(\text{spam} \mid \text{msg})}{P(\text{ham} \mid \text{msg})} = \log \frac{P(\text{spam})}{P(\text{ham})} + \sum_{w \in \text{msg}} \log \frac{P(w \mid \text{spam})}{P(w \mid \text{ham})}$$

Start from the prior log-odds and add, for every word in the message, its **log-likelihood ratio**: positive for spam-typical words, negative for ham-typical words. The final log-odds are turned into a probability with — the sigmoid. Naive Bayes in this form is a linear model in the word counts.

Table 14. Naive Bayes reading "WINNER! You have won a FREE prize. Call now to claim" word by word.

Word	Evidence added (log-ratio)	P(spam) so far
(prior)	-1.87	13.40%
winner	+3.46	83.08%
you	-0.85	67.62%

Word	Evidence added (log-ratio)	P(spam) so far
have	-0.22	62.64%
won	+2.20	93.80%
free	+2.34	99.36%
prize	+5.13	> 99.99%
call	+1.30	> 99.99%
now	+0.53	> 99.99%
to	+0.19	> 99.99%
claim	+5.43	> 99.99%

A second message, “Are we still meeting for lunch today? Call me when you are free”, contains the spam-typical words “call” and “free”, but “still”, “meeting”, “lunch”, “me” and “when” are strongly ham-typical and the final probability of spam is below 0.01%. Naive Bayes weighs all the evidence; no single word decides.

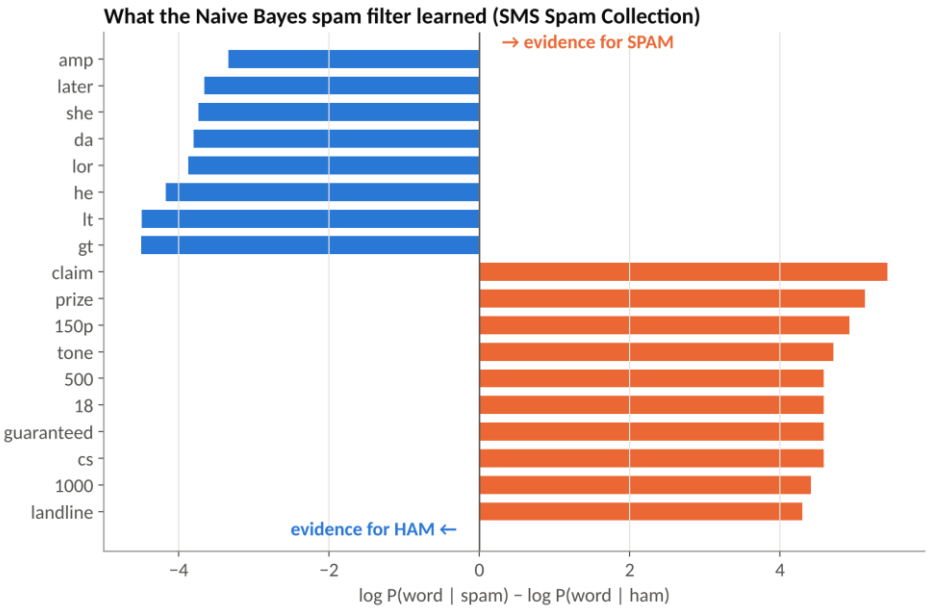


Figure 15. The words with the strongest evidence for spam (orange) and for ham (blue).

The spam words read like spam: claim, prize, guaranteed, awarded, 150p (a price per message), 1000, landline. The ham words are conversational: he, she, later, and local slang such as “lor” and “da”. Notice “lt”, “gt” and “amp”: these are fragments of HTML codes (< > &) from a placeholder that appears in many ham messages of this data set — an **artefact** of how the data were stored, not of language. Always inspect what your model learned; interpretable models make this easy.

Table 15. Results on the 1,393 test messages.

Model	Accuracy	Ham blocked (FP)	Spam missed (FN)
Naive Bayes (α = 1)	98.64%	2	17
Logistic regression	97.85%	0	30

Model	Accuracy	Ham blocked (FP)	Spam missed (FN)
Always “ham”	86.58%	0	187

Which model is better depends on the costs. A genuine message hidden in the spam folder usually costs the user much more than one extra spam in the inbox, so real filters choose their threshold to make false positives very rare.

5.9 Generative vs discriminative: who needs less data?

Ng and Jordan (2002) compared Naive Bayes and logistic regression in detail. Their result: logistic regression usually reaches a **lower error with enough data**, because it makes fewer assumptions, but Naive Bayes approaches its own best error with **far fewer training examples**, because its strong assumptions act as built-in knowledge.

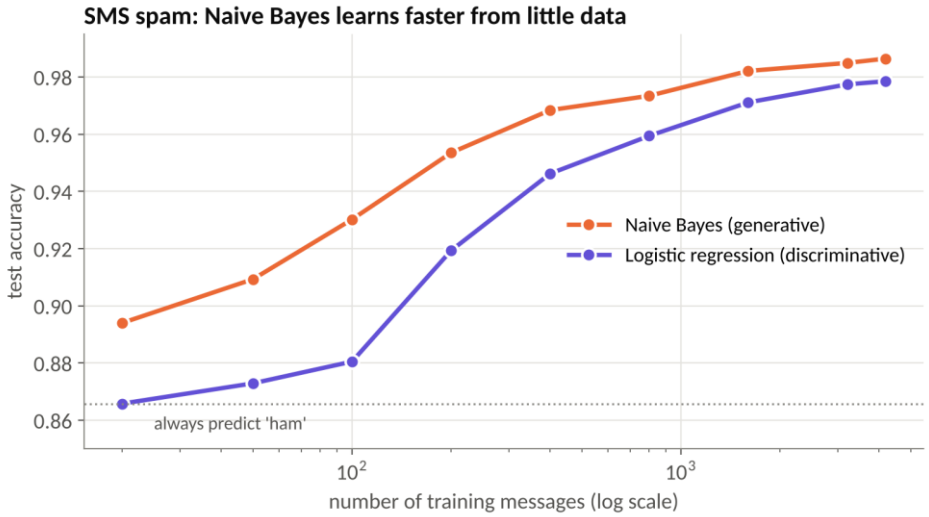


Figure 16. Test accuracy on the SMS data as a function of the number of training messages (average of 10 random subsets).

With only 100 training messages, Naive Bayes already reaches 93.0%, while logistic regression reaches 88.0% — barely above the always-ham baseline. The gap shrinks as the training set grows. This is a first look at next week's topic, the **bias–variance trade-off**: strong assumptions (high bias, low variance) win with little data; flexible models need more data but can go further.

TRY IT YOURSELF

A tiny filter has P(spam) = 0.2, P(‘win’ | spam) = 0.3, P(‘win’ | ham) = 0.01, P(‘today’ | spam) = 0.05, P(‘today’ | ham) = 0.1. Compute P(spam | ‘win today’).

6. Classification in practice

6.1 The models in scikit-learn

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, GaussianNB

# logistic regression (binary or multiclass - multiclass uses softmax automatically)
clf = make_pipeline(StandardScaler(), LogisticRegression(C=1.0))
clf.fit(X_train, y_train)
p = clf.predict_proba(X_test)[:, 1]        # probabilities of class 1
y_hat = (p >= 0.3).astype(int)             # your own threshold
odds_ratios = np.exp(clf[-1].coef_)        # per standard deviation

# Naive Bayes spam filter
spam = make_pipeline(CountVectorizer(), MultinomialNB(alpha=1.0))
spam.fit(messages_train, y_train)
spam.predict(["WINNER! Claim your prize"])  # array([1])

GaussianNB().fit(X_penguins, species)     # no scaling needed
```

Pipelines fit the scaler and the vocabulary on the training data only and reuse them on the test data, which prevents **data leakage**.

6.2 Where these models are used

- **Spam filtering.** In his 2002 essay “A Plan for Spam”, Paul Graham described a Bayesian filter that learned from each user's own mail and caught most spam with very few false positives. Within a year or two, Bayesian filtering was built into many mail programs and servers.
- **Online advertising.** McMahan et al. (Google, 2013) described click-through prediction with logistic regression trained online on billions of sparse features with L1-style regularisation. It is fast, scalable, gives meaningful probabilities for pricing ads, and can be updated continuously.
- **Credit scoring.** Many lenders use logistic-regression "scorecards", in which each feature contributes points (rescaled weights). In many countries a lender must be able to explain the main reasons for a rejection, which is easy with logistic regression.
- **Medicine.** Risk scores and diagnostic aids often use logistic regression because doctors need probabilities and coefficients they can check.

6.3 Logistic regression or Naive Bayes?

Table 16. Comparison of the two main models of this week.

	Logistic regression	Naive Bayes
Models	P(y | x) directly (discriminative)	P(x | y) and P(y) (generative)
Assumes	log-odds linear in x	feature distributions + independence given y
Little data	needs more examples	strong — learns fast

	Logistic regression	Naive Bayes
Lots of data	usually lower error	assumptions limit it
Training	iterative optimisation	one pass of counting
Probabilities	usually well calibrated	often too extreme — trust the ranking
Correlated features	shares weight between them	double-counts evidence
Feature scaling	needed (standardise)	not needed
Interpretability	weights = log odds ratios	word log-ratios

Practical advice: for text with little labelled data, start with Naive Bayes; for tabular data, start with regularised logistic regression; and always try both — they take seconds.

6.4 Common pitfalls

1. **Accuracy on imbalanced data.** With 13% spam, always-ham scores 87%; with one fraud per thousand payments, always-honest scores 99.9%. Compare with the baseline and look at the confusion matrix.
2. **The blind 0.5 threshold.** Choose the threshold from the costs, on validation data.
3. **Leakage.** Fit scalers, vocabularies and other preprocessing on the training data only — use pipelines.
4. **Unscaled features for logistic regression.** Gradient descent is slow and the penalty treats features unfairly. Standardise.
5. **Trusting Naive Bayes' probabilities.** 0.9999 from Naive Bayes does not mean 99.99%.
6. **No regularisation** on separable or high-dimensional data: exploding weights and overconfident probabilities.

6.5 Responsible use

Classifiers decide who receives a loan, which patient gets a second look, and which message you never see. Removing a protected attribute such as gender or ethnicity does **not** remove bias, because other features (postcode, names, shopping patterns) can act as **proxies**; outcomes and error rates must be measured for each group. People affected by a decision deserve an explanation. High overall accuracy can hide low recall for a rare group — exactly like the smallest class, Chinstrap, in Section 4.5. Data change over time (spammers adapt, customers change), so models must be monitored and retrained. For high-stakes decisions, a model's probability should support a professional's judgement, not replace it.

7. Summary and formula sheet

Six ideas to take home

1. **Probabilities first.** Predict P(y | x); the threshold, chosen from the costs of mistakes, turns it into a decision.
2. **Logistic regression:** $P(y = 1 \mid x) = \sigma(w^T x)$. The log-odds are linear, the boundary is a hyperplane, and e^(w_j) is an odds ratio.
3. **Maximum likelihood:** choose the parameters that make the observed data most probable; minimise the negative log-likelihood. Gaussian noise → squared error; Bernoulli labels → log loss.
4. **One gradient pattern:** X^T(prediction − truth) for linear, logistic and softmax regression. Logistic regression has no closed form → gradient descent; on separable data → regularise.
5. **Softmax** generalises the sigmoid to K classes, with the cross-entropy loss; compute it with the max-shift for stability.
6. **Naive Bayes:** Bayes' theorem + conditional independence; train by counting; smooth the counts; strong with little data, but overconfident probabilities.

Formula sheet

Quantity	Formula
Sigmoid and its derivative	$\sigma(z) = 1/(1 + e^{-z})$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
Logistic model	$P(y = 1 \mid x) = \sigma(w^T x)$; $\log(p/(1 - p)) = w^T x$
Decision boundary (t = 0.5)	$w^T x = 0$
Odds ratio	e^(w_j) per unit increase of x_j
Optimal threshold	t* = C_FP / (C_FP + C_FN)
Accuracy, recall, precision	(TP + TN)/n, TP/(TP + FN), TP/(TP + FP)
Likelihood (Bernoulli)	$L(w) = \Pi_i\, p_i^{y_i} (1 - p_i)^{(1-y_i)}$
Log loss	$J(w) = -(1/n)\, \Sigma_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$
Gradient (logistic)	$\nabla J = (1/n)\, X^T(\sigma(Xw) - y)$
L2-regularised loss	$J(w) + (\lambda/2)\|w\|^2$; scikit-learn $C = 1/\lambda$
MAP	$\hat{\theta} = \arg\max P(\text{data} \mid \theta)\, P(\theta)$; −log posterior = loss + penalty
Softmax	$P(y = k \mid x) = e^{(w_k^T x)} / \Sigma_j\, e^{(w_j^T x)}$
Cross-entropy and gradient	$J(W) = -(1/n)\, \Sigma_i \log p_{\{i,y_i\}}$; $\nabla J = (1/n)\, X^T(P - Y)$
Bayes' theorem	$P(y \mid x) = P(x \mid y)\, P(y) / P(x)$
Naive Bayes decision	$\hat{y} = \arg\max_k [\log P(y = k) + \Sigma_j \log P(x_j \mid y = k)]$
Gaussian NB likelihood	$P(x_j \mid y = k) = N(x_j; \mu_{jk}, \sigma^2_{jk})$

Quantity	Formula
Laplace smoothing	$P(w \mid c) = (\text{count}(w, c) + \alpha) / (\Sigma_v \text{count}(v, c) + \alpha\|V\|)$

8. Glossary

Term	Meaning
Base rate	The prior probability of a class in the population (e.g. disease prevalence).
Bag of words	Representation of a text by its word counts, ignoring order.
Bayes-optimal classifier	Predicts the class with the largest true posterior; the best possible classifier.
Cross-entropy	Negative log-likelihood of a categorical (or Bernoulli) model; the loss of softmax (logistic) regression.
Decision boundary	The set of inputs where the classifier switches from one class to another.
Discriminative model	Models P(y | x) directly (logistic, softmax regression).
False negative / positive	A missed positive / a false alarm.
Generative model	Models P(x | y) and P(y), and uses Bayes' theorem (Naive Bayes).
Laplace smoothing	Adding α imaginary counts to every word and class so that no probability is zero.
Likelihood	Probability of the observed data as a function of the parameters.
Log loss	−log of the probability given to the true class, averaged; binary cross-entropy.
Logit / log-odds	log(p/(1 − p)); the inverse of the sigmoid.
MAP estimate	Parameters that maximise likelihood × prior.
Maximum likelihood estimate	Parameters that maximise the likelihood.
Odds ratio	Factor e^(w_j) by which the odds change per unit increase of a feature.
One-hot encoding	A class k written as a vector with 1 in position k and 0 elsewhere.
One-vs-rest	Multiclass strategy with one binary classifier per class.
Precision / recall	Fraction of predicted positives that are correct / fraction of actual positives found.
Sigmoid	σ(z) = 1/(1 + e^(−z)); maps scores to probabilities.
Softmax	Maps K scores to K probabilities: exponentiate and normalise.
Threshold	Probability above which class 1 is predicted.

9. Practice exercises

Try each exercise on paper before reading the solution. Exercises 1–5 also answer the "Try it yourself" boxes.

Exercise 1. A standardised feature has weight w = −0.69. What happens to the odds of class 1 when the feature rises by one standard deviation?

Exercise 2. Compute the log loss of: (a) a spam message with P(spam) = 0.9; (b) a ham message with P(spam) = 0.4; (c) a ham message with P(spam) = 0.99.

Exercise 3. With 4 features plus a bias and 5 classes, how many parameters does softmax regression have? How many classifiers would one-vs-one need?

Exercise 4. Repeat the medical-test calculation of Section 5.2 for a prevalence of 10% (sensitivity 90%, false-positive rate 9%).

Exercise 5. P(spam) = 0.2, P('win' | spam) = 0.3, P('win' | ham) = 0.01, P('today' | spam) = 0.05, P('today' | ham) = 0.1. Compute P(spam | 'win today').

Exercise 6. Show that σ(−z) = 1 − σ(z), and that σ'(z) = σ(z)(1 − σ(z)).

Exercise 7. A coin shows 2 heads in 8 flips. Find the maximum-likelihood estimate of p, and the estimate with one imaginary head and one imaginary tail added.

Exercise 8. A logistic-regression model for a disease has log-odds = −3 + 0.05 × age. (a) What is the probability of disease at age 40? (b) At what age is the probability 0.5? (c) Interpret 0.05 as an odds ratio per 10 years.

Exercise 9. A fraud model has these test results: TN = 9,880, FP = 20, FN = 40, TP = 60. Compute accuracy, recall and precision. Is the model good?

Exercise 10. Missing a fraud costs €500; investigating a false alarm costs €25. Which threshold should the fraud model use?

Exercise 11. Softmax gives probabilities (0.7, 0.2, 0.1) for a penguin whose true species is the second class. What is its cross-entropy loss? What would it be if the model predicted (1/3, 1/3, 1/3)?

Exercise 12. Explain in two sentences why the maximum-likelihood weights of logistic regression do not exist when the classes are perfectly separable, and what scikit-learn does about it.

Exercise 13. Why does multinomial Naive Bayes need smoothing even with very large training sets?

Exercise 14. Which model would you try first, and why? (a) 60 labelled customer e-mails per category; (b) 2 million labelled loan applications with 40 numeric features, where reasons for rejection must be explained.

Solutions

1. The odds are multiplied by e^(−0.69) ≈ 0.50: they are halved. (The probability is not halved; how much it changes depends on its starting value.)

2. Use the probability of the **true** class. (a) −log 0.9 ≈ 0.105. (b) The truth (ham) got 0.6: −log 0.6 ≈ 0.51. (c) The truth got 0.01: −log 0.01 ≈ 4.61 — about 44 times the loss of (a). Confident mistakes dominate the average loss.

3. (4 + 1) × 5 = 25 parameters. One-vs-one: 5 × 4 / 2 = 10 classifiers.

4. Of 1,000 people, 100 are ill → 90 true positives; 900 healthy → 81 false positives. P(ill | positive) = 90/171 ≈ 0.53. The prior changes the answer from 9% to 53%.

5. Spam: 0.2 × 0.3 × 0.05 = 0.003. Ham: 0.8 × 0.01 × 0.1 = 0.0008. P(spam | message) = 0.003/(0.003 + 0.0008) ≈ 0.79. Odds view: prior odds ¼ × 30 (win) × ½ (today) = 3.75 ⇒ 3.75/4.75 ≈ 0.79.

6. σ(−z) = 1/(1 + e^z) = e^(−z)/(e^(−z) + 1) = 1 − 1/(1 + e^(−z)) = 1 − σ(z). For the derivative: σ′(z) = e^(−z)/(1 + e^(−z))² = [1/(1 + e^(−z))] · [e^(−z)/(1 + e^(−z))] = σ(z)(1 − σ(z)).

7. L(p) = p²(1 − p)⁶; d log L/dp = 2/p − 6/(1 − p) = 0 ⇒ p̂ = 2/8 = 0.25. With the prior counts: (2 + 1)/(8 + 2) = 0.3.

8. (a) log-odds = −3 + 2 = −1, p = σ(−1) ≈ 0.27. (b) −3 + 0.05 × age = 0 ⇒ age = 60. (c) Per year the odds are multiplied by e^0.05 ≈ 1.051; per 10 years by e^0.5 ≈ 1.65.

9. Accuracy = (9,880 + 60)/10,000 = 99.4%. Recall = 60/100 = 60%. Precision = 60/80 = 75%. The fraud rate is 1%, so always predicting "genuine" already gives 99.0% accuracy; the useful numbers are recall and precision. The model misses 40% of frauds — whether that is acceptable depends on the costs.

10. t* = C_FP/(C_FP + C_FN) = 25/(25 + 500) ≈ 0.048. Flag every payment with predicted fraud probability above about 5%.

11. −log 0.2 ≈ 1.61. With the uniform prediction: −log(1/3) = log 3 ≈ 1.10 — so this confident wrong prediction is worse than knowing nothing.

12. Scaling up the weights of a separating hyperplane keeps the boundary but pushes all training probabilities closer to the correct 0 or 1, so the log loss keeps decreasing towards 0 and there is no finite minimiser. scikit-learn adds an L2 penalty by default (C = 1), which keeps the weights finite.

13. The vocabulary is huge and most words are rare, so some words always have zero count in one class. A single zero makes P(message | class) = 0 and vetoes that class whatever the other words say; smoothing gives every word a small positive probability.

14. (a) Multinomial Naive Bayes (or strongly regularised logistic regression): with very little data, strong assumptions help. (b) Regularised logistic regression: plenty of data, numeric features, calibrated probabilities, and coefficients that can be explained.

10. Further reading and videos

- James, G., Witten, D., Hastie, T., Tibshirani, R. & Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*, Chapter 4 (§4.1–4.4). Free at statlearning.com.
- Jurafsky, D. & Martin, J. H. *Speech and Language Processing* (3rd ed. draft): the chapters on Naive Bayes and on logistic regression for text classification. Free at web.stanford.edu/~jurafsky/slp3.
- Géron, A. *Hands-On Machine Learning*, Chapter 4: logistic and softmax regression.
- Videos: StatQuest — “Logistic Regression”, “Maximum Likelihood, clearly explained!!!”, “Naive Bayes, Clearly Explained!!!”; 3Blue1Brown — “Bayes theorem, the geometry of changing beliefs”.
- Ng, A. Y. & Jordan, M. I. (2002). On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes. NIPS 14.
- Domingos, P. & Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. Machine Learning 29, 103–130.

- Gigerenzer, G. et al. (2007). Helping doctors and patients make sense of health statistics. Psychological Science in the Public Interest 8(2), 53–96.
- Graham, P. (2002). A Plan for Spam. paulgraham.com/spam.html. · McMahan, H. B. et al. (2013). Ad click prediction: a view from the trenches. KDD.
- Data: Gorman, Williams & Fraser (2014), Palmer penguins · Street, Wolberg & Mangasarian (1993), Wisconsin breast cancer · Almeida, Gómez Hidalgo & Yamakami (2011), SMS Spam Collection.